

实验四：自定义内存分配器 (Malloc) (学生版)

1. 实验目标

- 打破认知误解**：通过实现 `malloc`，理解其并不是系统调用，而是 C 标准库中的内存管理层。
- 理解堆管理**：掌握堆 (Heap) 的工作原理，学习如何利用 `sbrk` 扩展程序断点。
- 掌握链表管理**：实现隐式空闲链表 (Implicit Free List)，掌握元数据 (Metadata) 与用户数据的布局。
- 直观感受碎片**：了解内部碎片与外部碎片的成因。

2. 实验环境准备

- 操作系统**：建议使用 Ubuntu 22.04 LTS。
- 编译器**：`gcc` / `g++`。

3. 实验任务：实现一个简单的内存分配器

本任务要求你在 `my_malloc.c` 框架代码中补全 9 个关键的 `TODO`。我们将实现一个基于 **First-Fit (首次适配)** 策略的单向链表分配器。

框架代码详解

代码定义了一个 `block_meta` 结构体作为元数据，隐藏在每个分发给用户的内存块之前。

待补全代码 (`my_malloc.c`)

```
#include <unistd.h>
#include <string.h>
#include <stdio.h>

typedef struct block_meta {
    size_t size;
    int free; //空闲为1, 占用为0
    struct block_meta *next;
} block_meta;

#define META_SIZE sizeof(block_meta)
void *global_base = NULL;

// ===== 1. 寻找空闲块 =====
block_meta *find_free_block(block_meta **last, size_t size) {
    block_meta *current = global_base;
    // ★ TODO 1: 遍历链表, 使用 First-Fit 策略寻找满足条件的空闲块
    // 提示: 循环遍历 current, 直到找到 current->free == 1 且 size 足够。
    // 同时更新 *last, 使其始终指向当前遍历到的最后一个节点。

    return current;
```

```

}

// ===== 2. 向 OS 申请物理内存 =====
block_meta *request_space(block_meta *last, size_t size) {
    block_meta *block;

    // ★ TODO 2: 调用 sbrk(0) 获取当前堆顶，将它作为新 block 的起始地址。

    // ★ TODO 3: 调用 sbrk(META_SIZE + size) 扩展堆空间。
    // 检查返回值，如果为 (void*)-1 则申请失败，返回 NULL。

    // ★ TODO 4: 初始化新 block 的元数据: size、free、next。
    // 提示：新申请的块正在被使用，next 暂时没有后继。

    // ★ TODO 5: 如果 last 不为 NULL，将新 block 接到链表尾部。

    return block;
}

// ===== 3. 实现 my_malloc =====
void *my_malloc(size_t size) {
    if (size <= 0) return NULL;
    block_meta *block;

    if (!global_base) { // 第一次分配
        // ★ TODO 6: 第一次分配时，直接向 OS 申请空间，并初始化 global_base。
        // 提示：如果 request_space 返回 NULL，my_malloc 也应该返回 NULL。
    } else {
        block_meta *last = global_base;
        // ★ TODO 7: 调用 find_free_block 尝试寻找可复用的空闲块。
        // 如果返回 NULL，说明找不到合适空闲块，需要调用 request_space 扩容。
        // 如果找到了可复用块，只需要把它的 free 标记改为 0。
    }

    // ★ TODO 8: 返回给用户的指针应该是跳过头部元数据后的数据区起始地址。
    // 提示：(block + 1) 会根据结构体大小自动对齐偏移。

}

// ===== 4. 实现 my_free =====
void my_free(void *ptr) {
    if (!ptr) return;
    // ★ TODO 9: 根据用户传入的 ptr 反推出头部元数据地址。
    // 然后将 free 标记为 1。
    // 提示：想想 my_malloc 里是怎么做偏移的，反向操作。
}

```

4. 实验测试

在 `my_malloc.c` 中编写简单的 `main` 函数进行验证。这个测试只检查基础分配器能力，不涉及块分割、块合并等进阶功能：

```

int main() {
    printf("=== 测试 1: 基础分配与写入 ===\n");
    char *p1 = (char*)my_malloc(10);
    if (!p1) {
        printf("my_malloc 申请失败\n");
        return 1;
    }
    strcpy(p1, "OS_LAB");
    printf("p1 地址: %p, 写入内容: %s\n", p1, p1);

    printf("\n=== 测试 2: 释放后复用空闲块 ===\n");
    // 释放 p1 后, 再申请同样大小的空间。基础分配器应该能复用这块空闲内存。
    my_free(p1);
    char *p2 = (char*)my_malloc(10);
    printf("p2 地址: %p\n", p2);
    printf("是否复用了 p1 的地址: %s\n", (p2 == p1) ? "是" : "否");
    if (p2 != p1) {
        printf("【错误】p2 没有复用刚释放的 p1, 请检查 find_free_block 或 free 标记。\n");
        return 1;
    }
    strcpy(p2, "REUSE");
    printf("p2 写入内容: %s\n", p2);

    printf("\n=== 测试 3: 复用后的块必须重新标记为正在使用 ===\n");
    // 如果 my_malloc 复用 p1 后忘记把 p2 对应块的 free 改回 0,
    // 下面这次申请可能会错误地再次返回 p2 的地址。
    char *p3 = (char*)my_malloc(10);
    printf("p3 地址: %p\n", p3);
    printf("p3 是否没有错误复用正在使用的 p2: %s\n", (p3 != p2) ? "是" : "否");
    if (p3 == p2) {
        printf("【错误】p3 错误复用了正在使用的 p2, 请检查复用空闲块时是否执行 block->free = 0。 \n");
        return 1;
    }
    strcpy(p3, "NEW");
    printf("p3 写入内容: %s\n", p3);
    printf("p2 内容是否仍然保持为 REUSE: %s\n", strcmp(p2, "REUSE") == 0 ? "是" : "否");
    if (strcmp(p2, "REUSE") != 0) {
        printf("【错误】p2 的内容被覆盖, 说明两个指针可能指向了同一块内存。 \n");
        return 1;
    }

    printf("\n=== 测试 4: 正在使用的块不会被复用 ===\n");
    // p2 和 p3 都还没有被释放, 仍然是正在使用的块。再次申请内存时, 不应该复用它们的地址。
    char *p4 = (char*)my_malloc(100);
    printf("p4 地址: %p\n", p4);
    printf("p4 是否没有复用正在使用的 p2/p3: %s\n", (p4 != p2 && p4 != p3) ? "是" : "否");
    if (p4 == p2 || p4 == p3) {
        printf("【错误】p4 错误复用了正在使用的内存块, 请检查空闲块判断条件。 \n");
        return 1;
    }
    strcpy(p4, "larger allocation works");
    printf("p4 写入内容: %s\n", p4);
}

```

```

printf("\n=== 测试 5: 空闲块容量不足时不会被复用 ===\n");
// 释放 p2 后, 它会变成一个 10 字节空闲块。此时申请 100 字节, 不能复用这个小块。
my_free(p2);
char *p5 = (char*)my_malloc(100);
printf("p5 地址: %p\n", p5);
printf("p5 是否没有复用 10 字节的 p2: %s\n", (p5 != p2) ? "是" : "否");
if (p5 == p2) {
    printf("【错误】p5 错误复用了容量不足的 p2, 请检查 find_free_block 是否判断 current->size
    >= size。 \n");
    return 1;
}
strcpy(p5, "another larger allocation");
printf("p5 写入内容: %s\n", p5);

printf("\n=== 测试 6: 释放 NULL 指针 ===\n");
// free(NULL) 应该直接返回, 不能导致程序崩溃。
my_free(NULL);
printf("my_free(NULL) 执行完成, 程序没有崩溃\n");

// 清理测试中仍在使用的内存块。
my_free(p3);
my_free(p4);
my_free(p5);

printf("\n所有基础测试通过。 \n");
return 0;
}

```

编译并运行:

```
gcc my_malloc.c -o my_malloc
```

```
./my_malloc
```

运行结果:

```
● epoc@sophonlaptop:~/os_lab/lab4$ gcc my_malloc.c -o my_malloc
● epoc@sophonlaptop:~/os_lab/lab4$ ./my_malloc
=== 测试 1: 基础分配与写入 ===
p1 地址: 0x5b97dbc34018, 写入内容: OS_LAB

=== 测试 2: 释放后复用空闲块 ===
p2 地址: 0x5b97dbc34018
是否复用了 p1 的地址: 是
p2 写入内容: REUSE

=== 测试 3: 复用后的块必须重新标记为正在使用 ===
p3 地址: 0x5b97dbc3403a
p3 是否没有错误复用正在使用的 p2: 是
p3 写入内容: NEW
p2 内容是否仍然保持为 REUSE: 是

=== 测试 4: 正在使用的块不会被复用 ===
p4 地址: 0x5b97dbc3405c
p4 是否没有复用正在使用的 p2/p3: 是
p4 写入内容: larger allocation works

=== 测试 5: 空闲块容量不足时不会被复用 ===
p5 地址: 0x5b97dbc340d8
p5 是否没有复用 10 字节的 p2: 是
p5 写入内容: another larger allocation

=== 测试 6: 释放 NULL 指针 ===
my_free(NULL) 执行完成, 程序没有崩溃

所有基础测试通过。
```

5. 任务二：核心魔法——LD_PRELOAD 动态拦截

在完成任务一后，你已经拥有了一个可以工作的 `malloc`。现在，我们将使用 Linux 的 `LD_PRELOAD` 机制，强制让系统命令（如 `ls`）使用你编写的内存分配器。

5.1 编写拦截代码 `hook_malloc.c`

为了观察到拦截效果，我们需要编写一个包装层。

```
#define _GNU_SOURCE

#include <unistd.h>
#include <string.h>
#include <pthread.h>
#include <stdint.h>
```

```

#include <stddef.h>

/*
 * 必须和 my_malloc.c 中的 block_meta 保持完全一致。
 * 因为 realloc 需要通过用户指针 ptr 反推出 block_meta,
 * 然后读取 block->size, 知道旧块有多大。
 */
typedef struct block_meta {
    size_t size;
    int free; // 空闲为 1, 占用为 0
    struct block_meta *next;
} block_meta;

/*
 * 声明你在 my_malloc.c 中已经实现好的函数。
 * 注意: 这里不需要 my_calloc, 也不需要 my_realloc。
 */
void *my_malloc(size_t size);
void my_free(void *ptr);

static pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

/*
 * 拦截系统 malloc。
 */
void *malloc(size_t size) {
    const char msg[] = "Haha! I intercepted malloc!\n";
    write(STDOUT_FILENO, msg, sizeof(msg) - 1);

    pthread_mutex_lock(&lock);
    void *ptr = my_malloc(size);
    pthread_mutex_unlock(&lock);

    return ptr;
}

/*
 * 拦截系统 free。
 */
void free(void *ptr) {
    const char msg[] = "Haha! I intercepted free!\n";
    write(STDOUT_FILENO, msg, sizeof(msg) - 1);

    if (ptr == NULL) {
        return;
    }

    pthread_mutex_lock(&lock);
    my_free(ptr);
    pthread_mutex_unlock(&lock);
}

/*

```

```

* 拦截系统 calloc。
*
* calloc 的语义：
* 1. 分配 nmemb * size 字节
* 2. 将分配到的内存清零
*
* 这里底层仍然只使用你的 my_malloc。
*/
void *calloc(size_t nmemb, size_t size) {

    /*
     * 防止 nmemb * size 发生整数溢出。
     */
    if (nmemb != 0 && size > SIZE_MAX / nmemb) {
        return NULL;
    }

    size_t total = nmemb * size;

    pthread_mutex_lock(&lock);
    void *ptr = my_malloc(total);
    pthread_mutex_unlock(&lock);

    if (ptr != NULL) {
        memset(ptr, 0, total);
    }

    return ptr;
}

/*
* 拦截系统 realloc。
*
* realloc 的基本语义：
* realloc(NULL, size) 等价于 malloc(size)
* realloc(ptr, 0) 等价于 free(ptr)，然后返回 NULL
* 如果原块足够大，可以直接返回原指针
* 如果原块不够大，申请新块，复制旧数据，释放旧块
*
* 注意：
* 这里底层仍然只使用你的 my_malloc 和 my_free。
*/
void *realloc(void *ptr, size_t size) {

    if (ptr == NULL) {
        return malloc(size);
    }

    if (size == 0) {
        free(ptr);
        return NULL;
    }
}

```

```

block_meta *block = (block_meta *)ptr - 1;

/*
 * 如果旧块已经够大，直接复用。
 * 这个简化版 allocator 不做 split。
 */
if (block->size >= size) {
    return ptr;
}

/*
 * 旧块不够大，申请新块。
 * 注意这里调用 malloc，会进入上面我们自己拦截的 malloc，
 * 最终仍然走 my_malloc。
 */
void *new_ptr = malloc(size);
if (new_ptr == NULL) {
    return NULL;
}

/*
 * 只复制旧块大小的数据，防止越界读。
 */
memcpy(new_ptr, ptr, block->size);

/*
 * 释放旧块。
 * 这里调用 free，会进入我们自己拦截的 free，
 * 最终仍然走 my_free。
 */
free(ptr);

return new_ptr;
}

```

5.2 编译为共享库

执行以下命令，将你的代码编译成 `.so` 共享库：

```
gcc -shared -fPIC hook_malloc.c my_malloc.c -o hook_malloc.so -pthread
```

5.3 见证奇迹：截胡系统命令

使用 `LD_PRELOAD` 运行 `ls`：

```
LD_PRELOAD=./hook_malloc.so ls -al
```

预期现象：你会看到大量拦截提示和 `ls` 原本的目录输出混在一起，例如：

```
Haha! I intercepted malloc!
Haha! I intercepted malloc!
```



```
Haha! I intercepted free!
Haha! I intercepted realloc!
Haha! I intercepted calloc!
...
total 56
drwxr-xr-x 2 epoc epoc 4096 May 11 05:33 .
drwxr-xr-x 8 epoc epoc 4096 Apr 23 01:27 ..
-rw-r--r-- 1 epoc epoc 3597 May 11 05:33 hook_malloc.c
-rwxr-xr-x 1 epoc epoc 16464 May 11 05:33 hook_malloc.so
-rwxr-xr-x 1 epoc epoc 16320 May 11 05:25 my_malloc
-rw-r--r-- 1 epoc epoc 6970 May 11 05:25 my_malloc.c
Haha! I intercepted free!
```

这些输出说明：

1. `ls` 在真正打印目录内容之前，已经进行了很多动态内存申请和释放。
2. 真实程序不只使用 `malloc/free`，还可能使用 `realloc/calloc`。这些 `realloc/calloc` 提示来自 `hook_malloc.c` 中自己定义的同名拦截函数。
3. 输出提示和 `ls` 的目录列表混在一起，是因为拦截函数使用 `write(STDOUT_FILENO, ...)` 直接向标准输出写入内容，而 `ls` 自己也在向标准输出打印目录信息。
4. 程序结束前后仍然可能出现 `free` 提示，因为运行时库和程序清理阶段也会释放内部资源。

本实验给出的 `hook_malloc.c` 已经自己实现了 `calloc/realloc` 的拦截逻辑：

- `calloc` 内部调用 `my_malloc` 申请空间，然后使用 `memset` 清零。
- `realloc(NULL, size)` 会进入自己拦截的 `malloc`，最终调用 `my_malloc`。
- `realloc(ptr, 0)` 会进入自己拦截的 `free`，最终调用 `my_free`。
- 普通 `realloc(ptr, size)` 会根据旧块大小决定是否原地复用；如果需要新块，就通过自己拦截的 `malloc/free` 完成“申请新块、复制数据、释放旧块”。

因此，在这份代码中，`calloc/realloc` 并不是交给 `libc` 的原始实现处理，而是继续接到你自己的 `my_malloc/my_free` 分配器上。

6. 进阶挑战（选做题）

挑战一：块的分割（Splitting）

目前的实现在复用大空闲块时会造成浪费（例如：空闲块 100 字节，用户只要 10 字节，也全给用户）。

- **任务：**实现一个分割逻辑。如果找到的空闲块 `size > requested_size + META_SIZE + 1`，则将其切分为两个块。

挑战二：块的合并（Coalescing）

释放内存时，如果相邻的块也是空闲的，应该合并它们以减少碎片。

- **任务：**在 `my_free` 中通过遍历检查释放块的前后邻居并自动合并。

7. 实验思考

1. **元数据开销**：自定义分配器为什么不能只记录“用户申请了多少字节”，而必须在用户数据区之外维护额外元数据？
2. **堆空间管理**：本实验使用 `sbrk` 扩展堆空间。和每次都直接向操作系统申请一块全新内存相比，分配器自己维护空闲链表有什么意义？
3. **释放语义**：本实验中的 `my_free` 只把块标记为空闲，而不清空内容，也不立刻归还给操作系统。这样设计分别带来了哪些好处和风险？
4. **动态拦截范围**：真实程序可能调用 `malloc/free/calloc/realloc` 等多个接口。如果只替换其中一部分接口，可能会造成哪些不一致？
5. **真实分配器复杂性**：和本实验的简化分配器相比，生产环境中的 `malloc` 还需要重点解决哪些问题？请选择两点说明原因。

8. 实验提交

完成实验报告的填写，以“学号-姓名-操作系统实验四”命名

将实验报告提交到作业网站: <https://acm.scu.edu.cn/teach/>